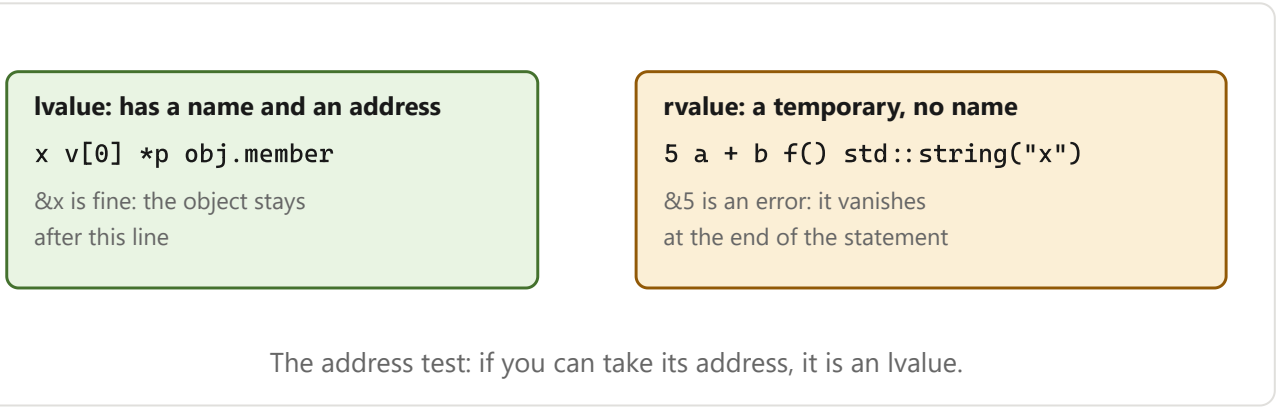
Lvalues and Rvalues

The idea underneath move semantics: which expressions are named objects and which are temporaries, the three value categories, and which references bind to what. Assumes Copy Control and Operators.

Names and temporaries

Every expression in C++ is either something with a **name and a place in memory** that lives on after the statement, or a **temporary** that is about to vanish. This split is the whole foundation of move semantics: a temporary is safe to steal from, because nobody can look at it afterwards.



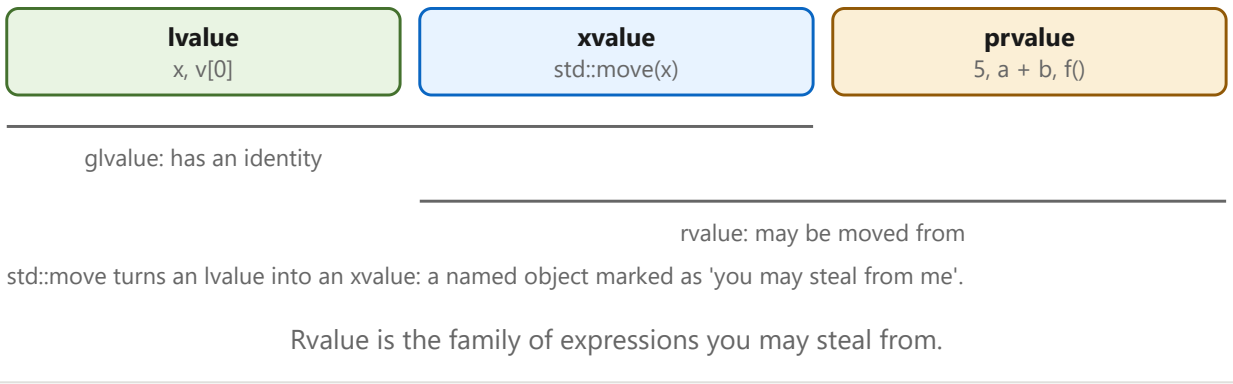
The address test: if you can take its address, it is an lvalue.

```
int x = 5;
int* p = &x;              // fine: x has an address
// int* q = &5;           // error: 5 is a temporary, it has no address
// int* r = &(x + 1);     // error: x + 1 is a temporary too
std::string a = "ab", b = "cd";
// &(a + b);              // error: the concatenation is a temporary
```

QUICK TEST Can you take its address with `&`? Then it is an lvalue. If the compiler refuses, it is an rvalue.

Lvalues, rvalues and value categories

Besides lvalue and rvalue there are three precise categories. You rarely need the names, but they explain the compiler messages.



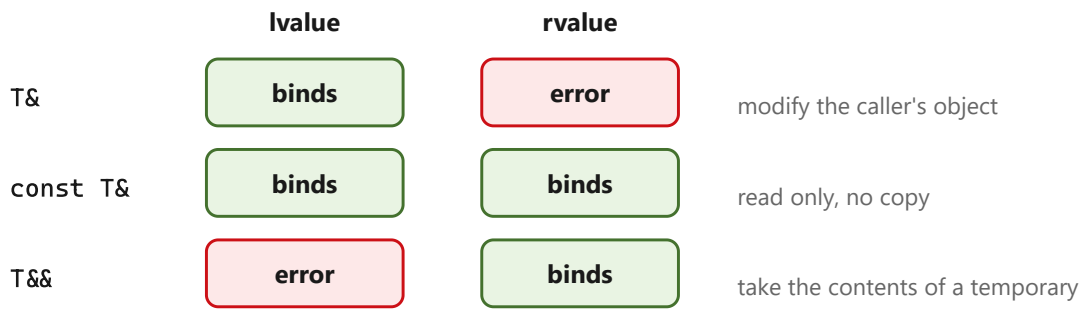
CATEGORY	ROUGHLY	EXAMPLES
lvalue	a named object	`x`, `v[0]`
prvalue	an unnamed temporary	`5`, `a + b`, a call that returns by value
xvalue	an object marked as movable	`std::move(x)`

You can ask the compiler which one an expression is: wrap the expression in a second pair of parentheses inside `decltype`.

```
int x = 0;
// lvalue → T&
static_assert(std::is_same_v<decltype((x)),           int&>);
// xvalue → T&&
static_assert(std::is_same_v<decltype((std::move(x))), int&&>);
// prvalue → T
static_assert(std::is_same_v<decltype((x + 1)),       int>);
```

Which reference binds to what

A reference type says which kinds of expression it accepts. That is how a function can *tell* whether it was given something it may steal from.



const T& binds to everything; T&& only to rvalues; T& only to lvalues.

```
// copy-or-read overload
void f(const std::string&) { std::cout << "const& "; }
// steal-from-a-temporary overload
void f(std::string&&)      { std::cout << "&& ";    }

std::string s = "a";
f(s);                   // const&   (s is an lvalue)
f(std::string("b"));    // &&       (a temporary)
f(std::move(s));        // &&       (std::move marks s as movable)
// prints: const& && &&   (compiled)
```

Overload resolution only looks at the *kind* of the argument: an rvalue prefers the `&&` overload, everything else takes the `const&` one. A move constructor is simply the `&&` overload of the copy constructor.

A name is always an lvalue

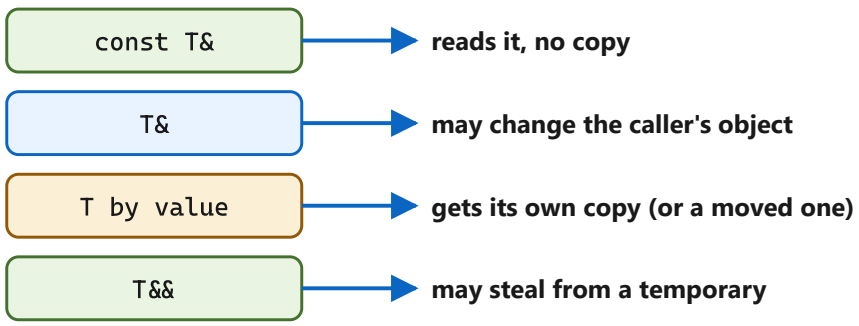
Inside a function that takes `std::string&& s`, the name `s` **is an lvalue**. The parameter can only *bind* to a temporary, but once it has a name you can use it twice, so it behaves like any named object.

```
void g(std::string&& s) {
    f(s);                // const&  : s has a name, so it is an lvalue
    f(std::move(s));     // &&      : mark it movable again
    std::cout << '\n';
}
g(std::string("x"));     // prints: const& &&   (compiled)
```

> **PASS IT ON EXPLICITLY** To hand a named rvalue reference to another function as an rvalue, write `std::move(s)` (or `std::forward` in a template, see Perfect Forwarding). Moving is always explicit in C++, so a name never gets stolen from by accident.

What a parameter type promises

Once you know the categories, a function signature becomes a short contract. This table is the whole story for ordinary parameters.



A signature is a promise: the parameter type says whether the function reads, changes, copies or steals.

PARAMETER	ACCEPTS	THE CALLER CAN ASSUME	USE FOR
`const T&`	lvalues and rvalues	the argument is unchanged and not copied	reading, the default for non-trivial types
`T&`	lvalues only	the argument may be modified	out-parameters (prefer a return value)
`T` (by value)	everything	the function has its own object	cheap types, or a sink that stores it
`T&&`	rvalues only	the argument will be left moved-from	move operations, rvalue overloads
`const T&&`	rvalues	(nothing useful)	almost never: it cannot be moved from

READING CODE When you see `std::move(x)` at a call, the rest of that function should not use `x` again. That is the convention that keeps moved-from objects harmless.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Names and temporaries	C++ Primer	13.6 Moving objects	560 to 579
	C++ Move Semantics	Chapter 2 Core features of move semantics	43 to 52
Lvalues, rvalues and value categories	C++ Primer	4.1 Fundamentals	163 to 167
	C++ Primer	13.6 Moving objects	560 to 579
	C++ Move Semantics	Chapter 2 Core features of move semantics	43 to 52
	C++ Move Semantics	8.1.2 to 8.1.3 and 8.4 to 8.5 Value categories since C++11 and C++17; when lvalues become rvalues and the reverse	145 to 148 and 153 to 154
	C++ Move Semantics	8.2 to 8.3 and 8.6 Special value-category rules; binding references; checking with decltype	148 to 153 and 154 to 156